Listing 8.8 Sample `toDataURL()` output

```
data:image/png;base64,iVBORw0KGgoAAAANSUhEUgAAAfQAAAH0CAYAAADL1t+KAAAgAElEQVR
4Xu2dC3xV1ZX/171B1JJggNoSsSY+QrWiQnB4dCoEH7Tgg4dVdNRCWg3SqQVm+i99TIfQmc7UPkbU
9sNDW0KVWluFYCl0FIdAW99AALWWUE2sCtWCgQQfkdz73+smV1NIyH3sc89ae//258
```

In our new example in the following listing, we create 100 circles of varying colors with varying borders. We then use `.toDataURL` to create an array of values that can be bound to `<img>` elements to create our first gallery of one hundred images.

Listing 8.9 Drawing 100 circles with canvas

```
imageArray = [];
  d3.select("#traditional").append("canvas")
               .attr("height", 500).attr("width", 500);

  var context = d3.select("canvas").node().getContext("2d");
  context.textAlign = "center";
  context.font="200px Georgia";
  colorScale = d3.scale.quantize().domain([0,1])
                          .range(colorbrewer.Reds[7]);

  lineScale = d3.scale.quantize().domain([0,1]).range([10,40]);
  for (var x=0;x<100;x++) {
     context.clearRect(0,0,500,500);
     context.strokeStyle = colorScale(Math.random());
     context.lineWidth = lineScale(Math.random());
     context.fillStyle = colorScale(Math.random());
     context.beginPath();
     context.arc(250,250,200,0,2*Math.PI);
     context.fill();
     context.stroke();

     context.fillStyle = "black";
     context.fillText(x,250,280);
     var dataURL = d3.select("canvas").node().toDataURL();
     imageArray.push({x: x, url: dataURL});
  }

  d3.select("#traditional")
  .append("div").attr("class", "gallery")
  .selectAll("img").data(imageArray)
  .enter().append("img")
  .attr("src", function(d) {return d.url})
  .style("height", "50px")
  .style("float", "left");
```

These scales are designed for random numbers to create random graphics.

Draws a randomly colored circle 100 times

Gets the data URL for each drawing and pushes it into an array

Uses that array to create 100 images

<img> elements have automatic resizing, so the width of the image automatically adjusts to scale the image to this height without distorting.

As shown in figure 8.8, each of our slightly different circles is turned into a PNG and assigned to an `<img>` element. We can also use `toDataURL()` to create JPEGs by specifying that format, but by default it creates PNGs. Because they're `<img>` elements now, they resize automatically. Even though we only specified the height of the images, the `<img>` element by default proportionately scaled the width of the image so that it wouldn't distort. Because of the `float:left` setting on those elements, they easily fill